

PLC Control Applications in CODESYS Validated Against a Factory I/O Digital Twin: Discrete, Sequential, and Analog Case Studies

Prajjwal Dutta
Arizona State University
Tempe, Arizona, USA
Email: pdutta9@asu.edu

Abstract—This report presents three programmable logic controller (PLC) applications developed in CODESYS V3.5 under IEC 61131-3 and commissioned against three-dimensional plant models running in Factory I/O, with the controller and the simulator coupled over an OPC DA/UA link. The three case studies were selected to span the classes of problem encountered in industrial control practice: a start/stop conveyor exercising discrete latching logic, a two-lane product sorting machine exercising sequential material handling with bistable state memory and event counting, and a process tank exercising analog signal conditioning between raw transmitter counts and engineering units. For each application we give the published process image, derive the control logic, and verify it against values sampled from the running system—tag traces for the two discrete applications and a three-point numerical check of both scaling transfer functions for the analog one. The results confirm correct latch behaviour with stop priority, speed-independent routing achieved by latching the classification at the point of measurement, and scaling accurate to within display rounding across the full measurement span. We argue that hardware-in-the-loop simulation exposes a class of timing-dependent defect that static review of the control program cannot.

Index Terms—Programmable logic controllers, ladder logic, IEC 61131-3, industrial automation, OPC, hardware-in-the-loop simulation, digital twin, process control.

I. INTRODUCTION

The programmable logic controller remains the dominant execution platform for discrete and hybrid industrial control, and IEC 61131-3 [1] remains the standard that defines its programming languages. Competence with the standard, however, is only partly a matter of language syntax. A control program that compiles cleanly may still fail on a plant that has actuator travel times, sensor response delays, and material inertia—failures that are intermittent, speed-dependent, and expensive to diagnose during commissioning.

This report documents three PLC applications written in CODESYS V3.5 [2] and validated end-to-end against 3D plant models in Factory I/O [3]. The three were chosen to form a progression across the principal problem classes:

- 1) **Discrete latching control** (Section III). A start/stop conveyor: the seal-in circuit that underpins every motor starter.
- 2) **Sequential material handling** (Section IV). A product sorting machine that classifies items by height and routes

them to one of two lanes, with bistable state memory and throughput counters.

- 3) **Analog signal conditioning** (Section V). A process tank whose level and flow transmitters are scaled into engineering units and displayed, with operator-driven valve control.

A. Contribution

The contribution of this work is methodological rather than novel in control theory. We show that (i) the memory element—whether expressed as a seal-in contact or as an explicit reset-dominant bistable—is the primitive from which the discrete and sequential applications are both built; (ii) correct sequential behaviour depends on placing that memory at the point of measurement rather than at the point of actuation; and (iii) the signal-conditioning layer of an analog loop can and should be verified numerically against live plant values before any regulator is layered on top of it. Each claim is supported by data read from the running system rather than by inspection of the source alone.

B. Why hardware-in-the-loop simulation

Simulation-based commissioning is well established in industrial practice, and Factory I/O in particular has been widely adopted for control-education and virtual-commissioning work [3], [5]. Its value here is specific: a race between a diverter command and a photo-eye, or a routing decision that depends on a sensor beam which has already cleared, appears only when the plant model supplies realistic dynamics. Such a defect is invisible to a code review and to a test bench that drives inputs synchronously. Placing a simulated plant in the loop moves that class of defect from commissioning into development.

II. SYSTEM ARCHITECTURE

All three applications share a single toolchain, shown in Fig. 1. The control logic executes in the CODESYS soft-PLC runtime. Factory I/O hosts the 3D plant model and exposes every simulated sensor and actuator as an OPC item. An OPC DA/UA server sits between them, so the controller never addresses the simulator directly—the same layering used when a physical PLC is commissioned against an HMI or SCADA tier [4].

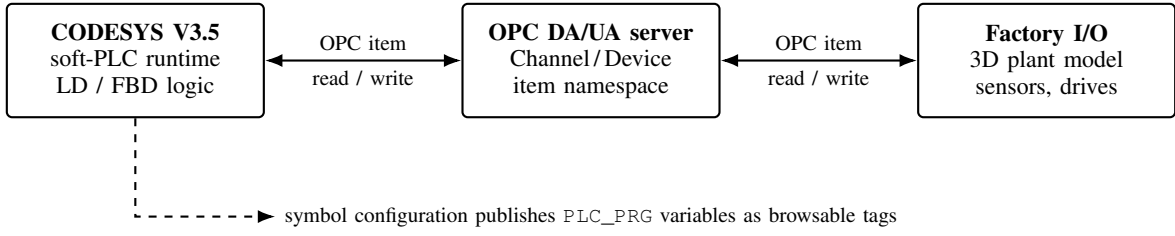


Fig. 1. Toolchain used for all three applications. The controller sees the plant purely as a set of OPC tags—the same abstraction it would have over a real fieldbus.

A. Process image

CODESYS exports the variables of the main program organisation unit `PLC_PRG` through a *symbol configuration* object. The OPC server browses that configuration and presents each variable as an item beneath a channel and device path, for example `Channel1.Device1.FIO_A` or `Channel2.Device1.Proj_05.FIO_5_Start`. Factory I/O, acting as an OPC client, binds each simulated sensor and actuator to one of those items. The symbol configuration is therefore the authoritative interface specification for each application, and the I/O tables in Sections III–V are taken directly from it.

B. Scan behaviour and latency

Each application runs a single cyclic task invoking `PLC_PRG`. Because the plant sits on the far side of an OPC link, the effective loop per scan is

$$\begin{array}{l} \text{PLC scan} \rightarrow \text{OPC write} \rightarrow \text{actuators} \\ \text{sensors} \rightarrow \text{OPC read} \rightarrow \text{PLC scan} \end{array}$$

The OPC round trip adds latency on top of the task period. This is directly observable in the recorded sessions as a small lag between a tag changing in the CODESYS watch table and the corresponding motion in the 3D scene. All three programs were therefore written to be robust to that delay: state is held in latches and counters rather than inferred from an instantaneous combination of sensor readings.

III. CASE STUDY I: DISCRETE LATCHING CONTROL

A. Problem statement

Drive a belt conveyor from a two-button station. A momentary *Start* must energise the drive and the drive must remain energised after the button is released; a *Stop* must break the circuit at any time and must take priority over *Start*. This is the seal-in, or hold-in, circuit—the smallest non-trivial memory element in ladder logic.

B. Process image

Table I lists the six variables published by the symbol configuration. All are of type `BOOL` with read/write access.

C. Control logic

The program comprises two independent ladder networks, redrawn in Fig. 3 and shown in online monitoring in Fig. 4.

TABLE I
CASE STUDY I — PROCESS IMAGE

Tag	Role	OPC item
A	Stop pushbutton	Channel1.Device1.FIO_A
B	Start pushbutton	Channel1.Device1.FIO_B
C	Conveyor drive	Channel1.Device1.FIO_C
AA	Exclusive-OR input 1	Channel1.Device1.FIO_AA
BB	Exclusive-OR input 2	Channel1.Device1.FIO_BB
CC	Exclusive-OR output	Channel1.Device1.FIO_CC

1) *Network 1 — seal-in latch*: In Boolean form,

$$C := \bar{A} \wedge (B \vee C). \quad (1)$$

Three properties of (1) are worth stating explicitly.

a) *Memory*: The `C` contact in the lower branch feeds the coil's own state back into the rung. Once `C` latches, that branch holds the rung true after the operator releases `B`. This is the entire mechanism by which a momentary button produces a maintained output.

b) *Stop priority*: The `A` term is in series *ahead* of the parallel block and is evaluated through a normally-closed contact. Whatever the state of `B` or of the seal, asserting `A` opens the rung; *Stop* can never be overridden by holding *Start*.

c) *Fail-safe convention*: Evaluating *Stop* through a normally-closed contact mirrors field wiring practice: a broken conductor to the *Stop* button de-energises the rung rather than disabling the stop function.

2) *Network 2 — exclusive-OR*: Ladder diagram provides no native exclusive-OR contact, so the function is composed from two parallel paths, each requiring one input asserted and the other clear:

$$CC := (AA \wedge \bar{BB}) \vee (\bar{AA} \wedge BB). \quad (2)$$

The two paths are mutually exclusive by construction, so the coil is energised for exactly one asserted input.

D. Verification

Table II gives tag states sampled from the OPC client across a complete cycle. The `C` column is the direct evidence for the seal: `B` reads 0 in every sampled frame, because the *Start* press is momentary and shorter than the sampling interval, yet `C` holds at 1. The latch, not the button, is maintaining the drive. Asserting `A` at the end of the run breaks the rung as (1) requires. Throughout, `CC` remains 0 because `AA` and `BB` are both asserted, which is the correct result of (2).

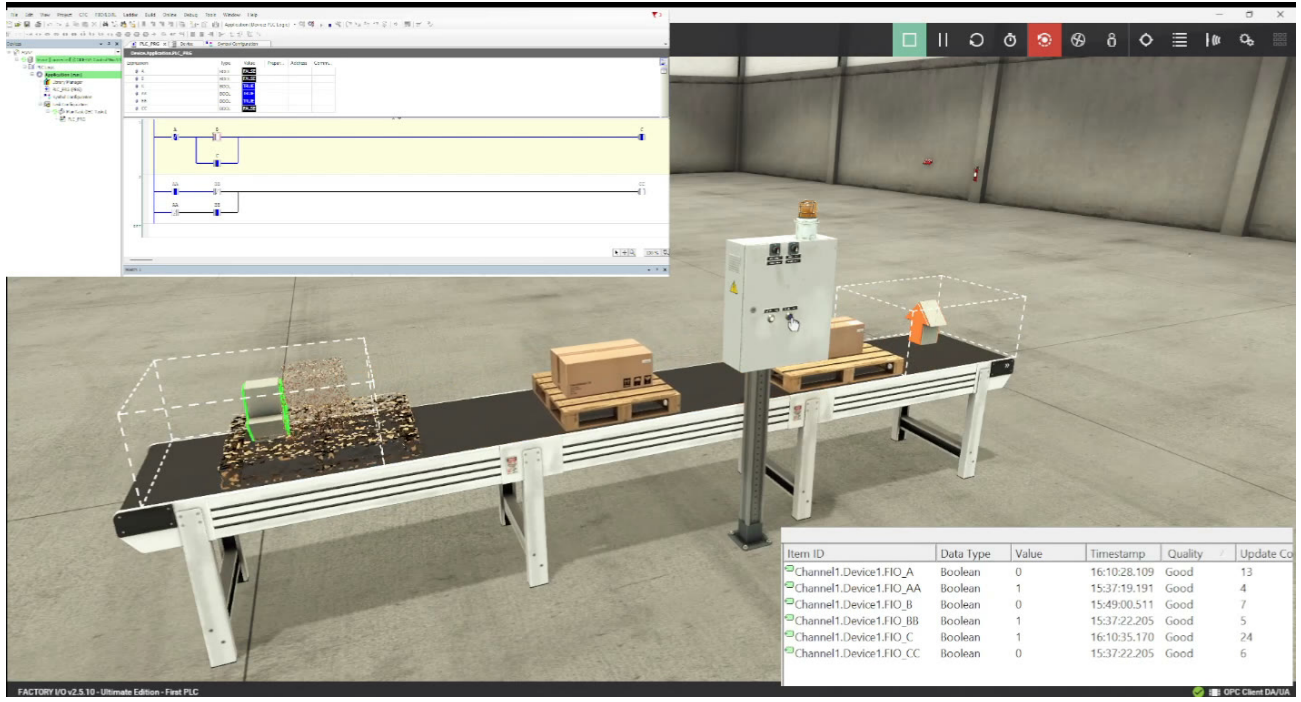


Fig. 2. Commissioning view for Case Study I: the CODESYS ladder editor in online mode (top left), the Factory I/O conveyor scene, and the live OPC tag table (bottom right) showing all six Boolean items with *Good* quality and incrementing update counts.

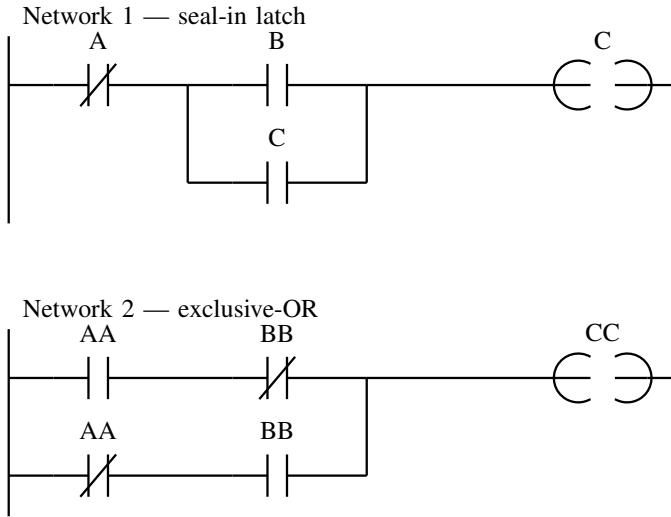


Fig. 3. The two ladder networks of Case Study I, redrawn from the online view. Network 1 seals the drive on through its own contact; Network 2 composes an exclusive-OR from two mutually exclusive parallel paths.

E. Discussion

This implementation latches through the output coil's own contact rather than through an RS function block. Both forms are valid under IEC 61131-3. The contact form is closer to the relay circuit it replaces and is what maintenance personnel typically expect to find; the block form, used in Case Study II, is preferable when several independent states must be held simultaneously and explicit set and reset conditions read more clearly than a network of seal contacts. The distinction is not



Fig. 4. The two networks in CODESYS online monitoring. Blue conductors carry power and filled contacts are conducting. Network 1 is sealed in through C; Network 2 is blocked because both exclusive-OR inputs are asserted.

TABLE II
CASE STUDY I — SAMPLED TAG TRACE

Phase	A	B	C	AA	BB	CC	State
Initial	0	0	0	1	1	0	belt stopped
After Start	0	0	1	1	1	0	latched, button released
After Stop	1	0	1 → 0	1	1	0	rung broken

purely stylistic: reset dominance is explicit in RS and implicit in rung ordering when seal contacts are used.

IV. CASE STUDY II: SEQUENTIAL MATERIAL HANDLING

A. Problem statement

Load a box onto a pallet, classify it by height, and divert it into one of two outfeed lanes; count the items dispatched and display the total. The station must support start, stop, reset and emergency-stop from an operator panel.

B. Plant layout

The plant is a T-shaped roller-conveyor station. A box is placed on a pallet by the loader, commanded by Load

Item ID	Data Type	Value	Timestamp	Quality	Update Co
Channel1.Device1.FIO_A	Boolean	0	16:10:28.109	Good	13
Channel1.Device1.FIO_AA	Boolean	1	15:37:19.191	Good	4
Channel1.Device1.FIO_B	Boolean	0	15:49:00.511	Good	7
Channel1.Device1.FIO_BB	Boolean	1	15:37:22.205	Good	5
Channel1.Device1.FIO_C	Boolean	1	16:10:35.170	Good	24
Channel1.Device1.FIO_CC	Boolean	0	15:37:22.205	Good	6

Fig. 5. OPC client item list showing data type, live value, timestamp, quality and update count. Rising update counts on A and C confirm the link is live rather than serving cached values.

and acknowledged by `Loaded`. The pallet then travels along the entry conveyor past a pair of photo-eyes mounted at different heights: `Low_sensor` breaks for every item, while `High_sensor` breaks only for tall ones. The combination classifies the item before it reaches the junction, where one of two transfer units lifts the pallet and drives it into the corresponding lane. Photo-eyes at the entry and exit of each lane confirm the hand-off.

C. Process image

Table III lists all nineteen published variables. All are `BOOL` except `Digital_display`, which is a `WORD`.

D. Control logic

The program is constructed from standard IEC 61131-3 function blocks rather than from bare seal contacts. Online monitoring of `PLC_PRG` exposes the instance tree summarised in Table IV: five RS bistables and two CTUD counters.

Two design decisions in this structure merit comment.

1) *Reset-dominant bistables*: RS is the reset-dominant bistable of IEC 61131-3: when Set and Reset are asserted in the same scan, Reset prevails. For a sorting station this is the safe polarity. Should a stop or emergency condition coincide with a set condition, the affected state clears rather than latching a transfer unit on.

2) *Latched classification*: `Box_type` is held in `RS_2` rather than read directly at the diverter. The height sensors are mounted upstream of the junction, so by the time the pallet arrives the beams have already cleared and a live read at the point of actuation would always fail. Latching the classification at the instant of measurement, and holding it until the transfer completes, is what makes the routing decision independent of conveyor speed. This is the central point of the case study: in a sequential application the memory must be placed at the point of measurement, not the point of actuation.

E. Operator interface

Fig. 8 shows the control panel: an emergency-stop mushroom head, Start and Stop pushbuttons, a Reset pushbutton, a mode selector, and the digital display driven by `Digital_display`.

F. Verification

Across the recorded session the station cycles continuously. Table V summarises the tag states characterising each phase.

Counting is triggered on the lane *exit* sensor rather than the entry sensor. An item is therefore credited only once it has

physically cleared the lane, so a pallet that stalls mid-transfer is not counted—a small choice with a direct effect on the integrity of the throughput figure.

G. Discussion

This case study makes the distinction between combinational and sequential logic concrete. No part of the routing decision can be computed from the instantaneous sensor vector: the height was measured seconds earlier and metres upstream. The RS instances are the program's memory of that measurement and the CTUD instances its memory of throughput. An error in either produces an intermittent, speed-dependent fault—precisely the class of defect that simulation catches and static review does not.

V. CASE STUDY III: ANALOG SIGNAL CONDITIONING

A. Problem statement

Control the fill and discharge valves of a process tank from two operator potentiometers, and display the measured level and volumetric flow in engineering units. Unlike the two discrete case studies, every signal here is analog, so the engineering effort lies in scaling rather than in Boolean composition.

B. Process image

Table VI lists the eight published variables. The intermediates `WaterLevel` and `FlowMeter` are internal `REAL` values and are deliberately not exported to the OPC server, since they carry no meaning outside the controller.

C. Control logic

The program comprises four function block diagram networks, summarised in Table VII and shown in Fig. 11.

1) *Fill valve*: The fill potentiometer and the valve share the same 0...10 range, so no conversion is required and `MOVE` transfers the value each scan.

2) *Discharge valve*: The discharge potentiometer is a centre-detent device reading -5 at one stop and $+5$ at the other. `FC_Scaling` maps that bipolar span onto the valve's unipolar travel by the standard linear transform

$$y = \frac{x - x_{\min}}{x_{\max} - x_{\min}} (y_{\max} - y_{\min}) + y_{\min}, \quad (3)$$

which for this network gives

$$\text{Tank1_DischargeValve} = \frac{x + 5}{10} \times 10. \quad (4)$$

At the full-clockwise position, $x = 5$ yields 10—the value observed in the recorded session. Admitting a bipolar input allows a single centre-detent control to sweep the valve across its full travel.

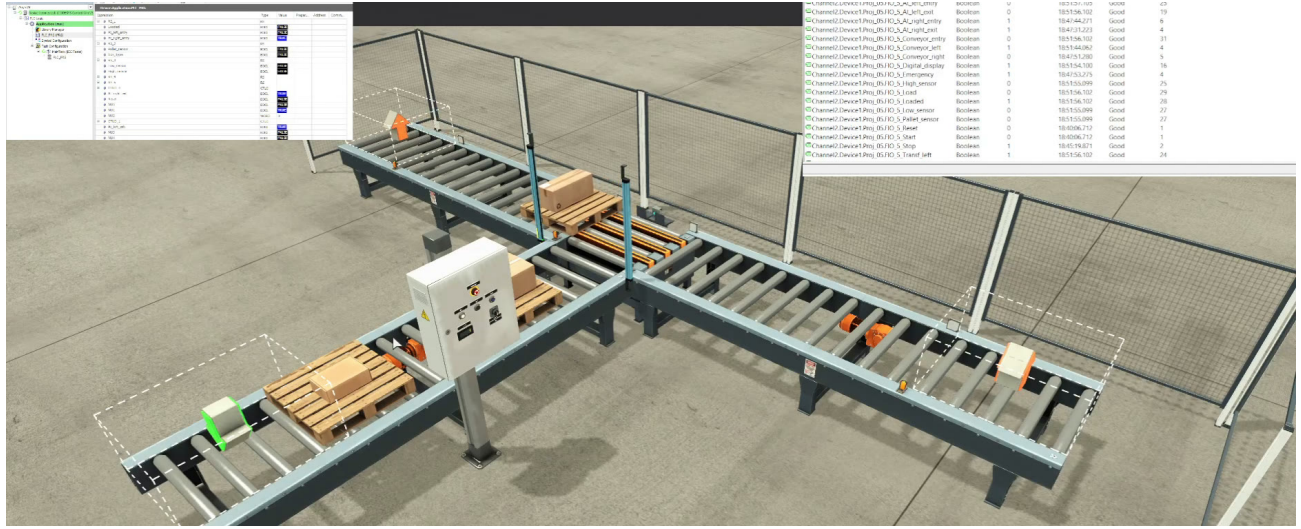


Fig. 6. Case Study II station layout: entry conveyor with pallet loader (upper left), the transfer junction at centre, and the left and right outfeed lanes.

TABLE III
CASE STUDY II — PROCESS IMAGE (19 TAGS)

Tag	Type	Dir.	Function
Start	BOOL	in	Start pushbutton
Stop	BOOL	in	Stop pushbutton
Reset	BOOL	in	Reset pushbutton; clears the counters
Emergency	BOOL	in	Emergency stop
Pallet_sensor	BOOL	in	Pallet present at the loading position
Loaded	BOOL	in	Loader hand-off complete
Low_sensor	BOOL	in	Low photo-eye; asserts for any item
High_sensor	BOOL	in	High photo-eye; asserts for tall items only
At_left_entry	BOOL	in	Item has entered the left lane
At_left_exit	BOOL	in	Item has cleared the left lane
At_right_entry	BOOL	in	Item has entered the right lane
At_right_exit	BOOL	in	Item has cleared the right lane
Load	BOOL	out	Command to the pallet loader
Conveyor_entry	BOOL	out	Entry conveyor drive
Conveyor_left	BOOL	out	Left lane conveyor drive
Conveyor_right	BOOL	out	Right lane conveyor drive
Transf_left	BOOL	out	Left transfer unit
Transf_right	BOOL	out	Right transfer unit
Digital_display	WORD	out	Panel counter value

TABLE IV
CASE STUDY II — FUNCTION BLOCK INSTANCES

Instance	Type	Purpose
RS_1	RS	Loader handshake and lane occupancy
RS_2	RS	Latches the height classification (Box_type)
RS_3	RS	Height-sensor pair conditioning
RS_4, RS_5	RS	Additional sequence state
CTUD_0	CTUD	Lane counter; WORD accumulator to the display
CTUD_1	CTUD	Lane counter, reset by Reset

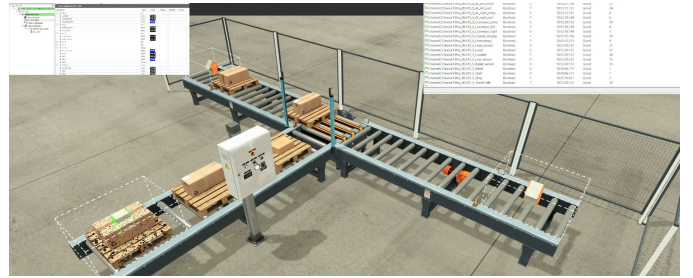


Fig. 7. Two pallets in the station: one waiting on the entry conveyor and one already diverted. The occupancy latches prevent the entry conveyor from feeding a second item into an occupied junction.

3) *Level and flow indication:* Networks 3 and 4 apply (3) in the opposite direction, converting a raw 0...10 transmitter

signal into the units printed on the tank's gauge board before TO_INT truncates the result to the display's integer type. Each

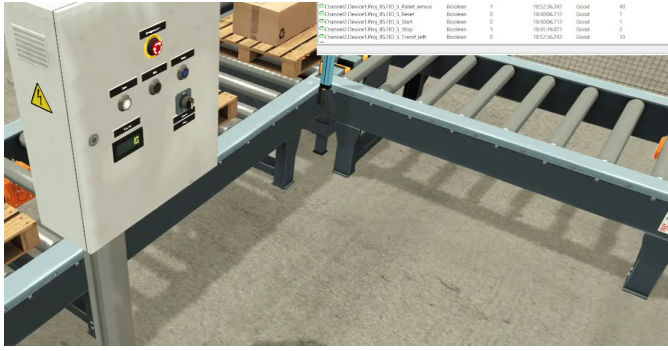


Fig. 8. Operator panel with emergency stop, Start, Stop and Reset pushbuttons, a mode selector, and the counter display.

TABLE V
CASE STUDY II — OBSERVED CYCLE PHASES

Phase	Key tags	Interpretation
Idle	Stop=1, Conveyor_entry=0	Awaiting Start
Infeed	Conveyor_entry=1, Pallet_sensor=1	Item advancing to junction
Classify	Low_sensor=1, High_sensor=0/1	Height measured and latched
Divert	Transf_left=1, At_left_entry=1	Transfer driving into lane
Tally	Digital_display increments	Counter fires on lane exit

FC_Scaling instance also drives an Input_Error output (Error1–Error3) that asserts if the input falls outside the declared span. All three read FALSE throughout the session, confirming that the transmitters remained within their configured ranges.

D. Verification

Both transfer functions were checked against live values sampled from the online view at three points across the operating range. Table VIII gives the results.

Both transfer functions match the expected values to within display rounding across the full span, confirming the FC_Scaling parameters of Table VII. The TO_INT truncation is also visible in the data: a level of 0.0997 presents as 0 and a flow of 18.7 presents as 19.

E. Discussion

This application is deliberately open-loop: the operator closes the loop by observing the displays and adjusting the potentiometers. No PID block is present and none is claimed. That is the correct starting point, because the signal-conditioning layer must be verified before any regulator built on top of it can be trusted. A controller tuned against a mis-scaled transmitter will appear stable while holding the wrong setpoint—an error that is invisible in the controller’s own diagnostics.

The natural extension is to replace networks 1 and 2 with a PID block driving Tank1_FillValve from the error

Item ID	Data Type	Value	Timestamp	Quality	Update Count
Channel2.Device1.Proc.BS_IO_5_AI_Left_Entry	Boolean	0	18:52:21.168	Good	31
Channel2.Device1.Proc.BS_IO_5_AI_Left_Exit	Boolean	1	18:52:11.151	Good	24
Channel2.Device1.Proc.BS_IO_5_AI_Right_Entry	Boolean	1	18:52:12.161	Good	8
Channel2.Device1.Proc.BS_IO_5_AI_Right_Exit	Boolean	1	18:52:18.164	Good	6
Channel2.Device1.Proc.BS_IO_5_Conveyor_Entry	Boolean	0	18:52:20.152	Good	39
Channel2.Device1.Proc.BS_IO_5_Conveyor_Left	Boolean	1	18:52:16.148	Good	6
Channel2.Device1.Proc.BS_IO_5_Conveyor_Right	Boolean	0	18:52:18.164	Good	7
Channel2.Device1.Proc.BS_IO_5_Digital_Display	Boolean	1	18:52:15.166	Good	20
Channel2.Device1.Proc.BS_IO_5_Emergency	Boolean	1	18:45:19.871	Good	4
Channel2.Device1.Proc.BS_IO_5_High_Sensor	Boolean	0	18:52:20.152	Good	31
Channel2.Device1.Proc.BS_IO_5_Load	Boolean	0	18:52:20.152	Good	37
Channel2.Device1.Proc.BS_IO_5_Loaded	Boolean	1	18:52:20.152	Good	36
Channel2.Device1.Proc.BS_IO_5_Loaded	Boolean	0	18:52:20.152	Good	35
Channel2.Device1.Proc.BS_IO_5_Pallet_Sensor	Boolean	0	18:52:20.152	Good	35
Channel2.Device1.Proc.BS_IO_5_Reset	Boolean	0	18:40:06.712	Good	1
Channel2.Device1.Proc.BS_IO_5_Start	Boolean	0	18:40:06.712	Good	1
Channel2.Device1.Proc.BS_IO_5_Stop	Boolean	1	18:45:19.871	Good	2
Channel2.Device1.Proc.BS_IO_5_Transf_Left	Boolean	1	18:52:20.152	Good	30

Fig. 9. All nineteen items live on the OPC server. Update counts differ per tag, separating signals that cycle with every pallet from static operator inputs.

TABLE VI
CASE STUDY III — PROCESS IMAGE

Tag	Type	Function
Pot1_Fill	REAL	Operator potentiometer, fill demand
Pot2_Discharge	REAL	Operator potentiometer, discharge demand
Tank1_LevelMeter	REAL	Level transmitter, raw 0...10
Tank1_FlowMeter	REAL	Flow transmitter, raw 0...10
Tank1_FillValve	REAL	Fill valve position
Tank1_DischargeValve	REAL	Discharge valve position
Display1_WaterLevel	INT	Panel display, level
Display2_FlowRate	INT	Panel display, flow rate

between a setpoint and WaterLevel, retaining the discharge valve as a manually injected disturbance. Networks 3 and 4 would carry over unchanged, which is precisely the benefit of separating signal conditioning from control law.

VI. COMPARATIVE DISCUSSION

Table IX summarises the three case studies.

Four observations follow from the set.

1) *Latching is the primitive*: The seal-in contact of Case Study I and the RS blocks of Case Study II express the same idea in two notations. Selecting between them is largely a readability and maintainability decision—with the substantive exception of reset dominance, which RS states explicitly and a seal contact leaves implicit in rung ordering.

2) *Memory belongs at the point of measurement*: Case Study II’s height classification cannot be re-derived downstream. Latching it when the sensor fires is what decouples the routing logic from conveyor speed, and is the difference between a program that works at any line rate and one that works only at the rate it was tested at.

3) *Scaling is a first-class concern*: Case Study III shows that most of the engineering in an analog loop occurs before any control law is applied, and that the scaling can be verified numerically against live plant values rather than assumed correct.

4) *Simulation closes the loop on correctness*: Every claim in this report is supported by values read from the running system: tag traces for the discrete applications and a three-point numerical check of both transfer functions for the analog one. This is only possible because a plant model was in the loop.

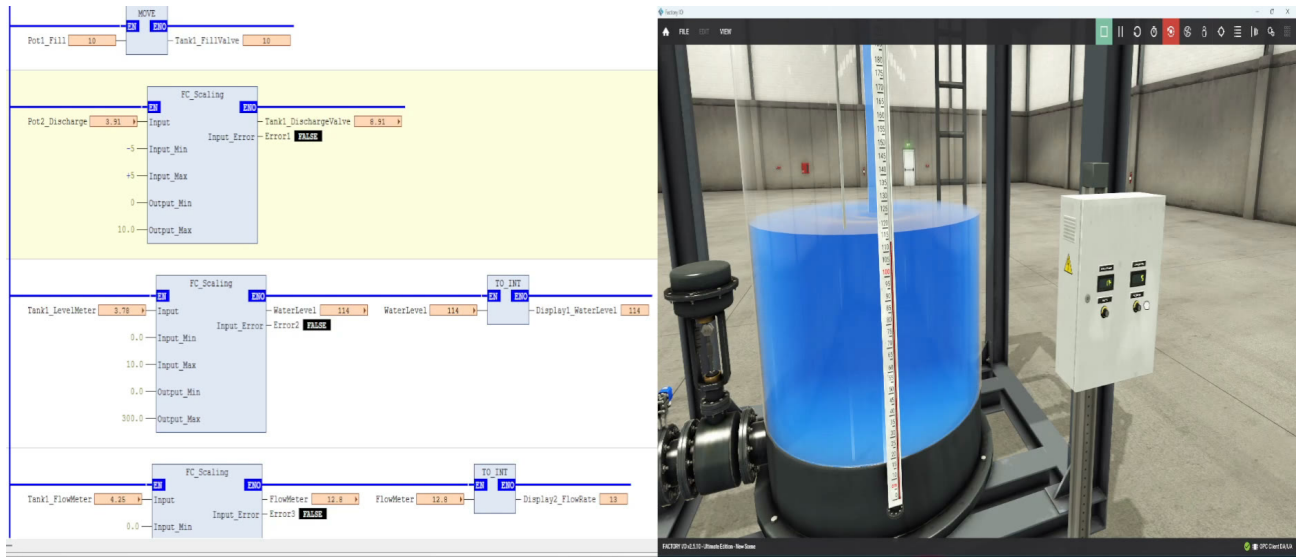


Fig. 10. Case Study III: the function block diagram program in online monitoring (left) beside the Factory I/O tank scene (right). Live values appear in the boxes attached to each pin.

TABLE VII
CASE STUDY III — FUNCTION BLOCK DIAGRAM NETWORKS

#	Blocks	Source	Input span	Output span	Destination
1	MOVE	Pot1_Fill	pass-through		Tank1_FillValve
2	FC_Scaling	Pot2_Discharge	-5.0... + 5.0	0.0...10.0	Tank1_DischargeValve
3	FC_Scaling, TO_INT	Tank1_LevelMeter	0.0...10.0	0.0...300.0	Display1_WaterLevel
4	FC_Scaling, TO_INT	Tank1_FlowMeter	0.0...10.0	0.0...30.0	Display2_FlowRate

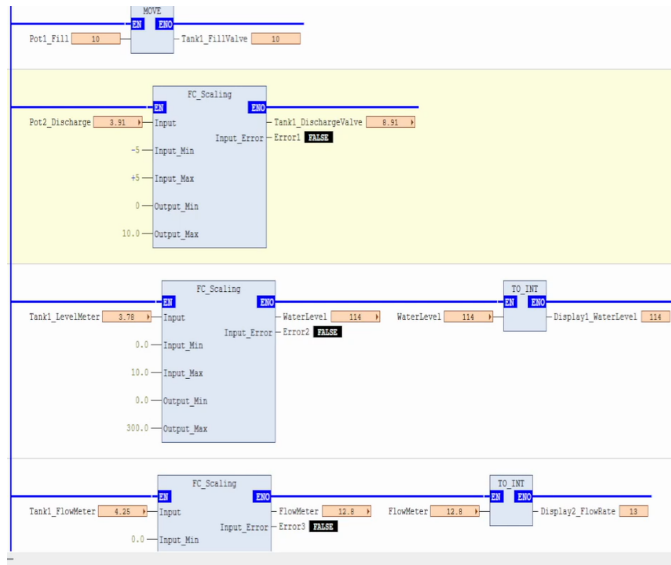


Fig. 11. The four function block diagram networks in online monitoring. Network 1 passes the fill demand through; network 2 rescales the bipolar discharge demand; networks 3 and 4 convert raw transmitter signals into engineering units and truncate them for display.

VII. LIMITATIONS

Three limitations should be stated.

TABLE VIII
CASE STUDY III — NUMERICAL VERIFICATION OF BOTH TRANSFER FUNCTIONS

Level: Tank1_LevelMeter ×30			
Raw	Expected	Observed	Display (INT)
3.86	115.8	116	116
3.58	107.4	107	107
0.00337	0.101	0.0997	0
Flow: Tank1_FlowMeter ×3			
Raw	Expected	Observed	Display (INT)
2.62	7.86	7.87	8
5.98	17.94	17.9	18
6.22	18.66	18.7	19

First, Case Study III implements no automatic level regulation; the loop is closed by the operator, and the scaling layer alone is verified.

Second, the rung-level implementation of Case Study II is documented from its online instance tree and from observed plant behaviour. The complete ladder was not captured in the recorded session, so the individual rungs are not reproduced here; the CODESYS project file accompanying this report is the authoritative source.

Third, the CODESYS project files are stored in a proprietary container format. They are not diff-friendly under version

TABLE IX
COMPARISON OF THE THREE CASE STUDIES

	I — Conveyor	II — Sorting	III — Tank
Language	Ladder (LD)	Ladder (LD)	Function Block Diagram (FBD)
Problem class	Discrete, combinational + latch	Sequential material handling	Analog signal conditioning
Process image	6 BOOL	18 BOOL + 1 WORD	6 REAL + 2 INT
Key constructs	Seal-in contact, NC stop, XOR	RS $\times 5$, CTUD $\times 2$	MOVE, FC_Scaling $\times 3$, TO_INT $\times 2$
Memory strategy	Output coil self-seal	Explicit reset-dominant bistables	Stateless per-scan transform
Validation method	Tag trace through latch and stop	Full sort cycles with counter tally	Three-point numerical check

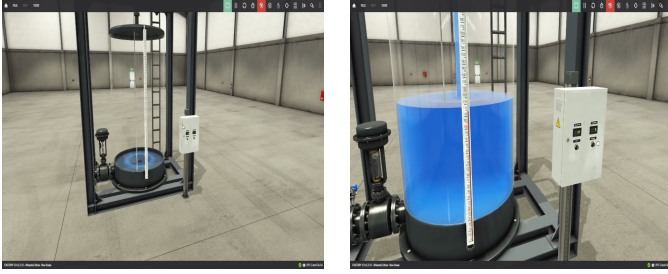


Fig. 12. The tank at the start of the run (left) and after filling (right). The graduated column and the two panel displays track together, verifying the scaling end-to-end against the plant model.

control, which limits the usefulness of conventional source-control review for this class of artefact.

VIII. CONCLUSION

Three PLC applications spanning discrete, sequential and analog control were implemented in CODESYS under IEC 61131-3 and commissioned against 3D plant models in Factory I/O over an OPC DA/UA link. The discrete application demonstrates a seal-in latch with correct stop priority, confirmed by a tag trace in which the output holds while the start input reads zero. The sequential application demonstrates speed-independent routing achieved by latching the height classification at the point of measurement in reset-dominant bistables, with throughput counted on lane-exit confirmation. The analog application demonstrates linear scaling between raw transmitter signals and engineering units, verified numerically at three points on each of two transfer functions and matching to within display rounding.

Taken together, the three establish that the memory element is the shared primitive of discrete and sequential control, that its placement relative to the measurement determines whether the program is robust to plant timing, and that the signal-conditioning layer of an analog loop is independently verifiable and should be verified before any regulator is added. The practical conclusion is that a plant model in the loop is not a convenience but a verification instrument: it makes claims about control behaviour testable against data rather than against inspection.

APPENDIX A TOOLCHAIN VERSIONS

Component	Version
CODESYS Development System	V3.5 SP22 Patch 2+
CODESYS runtime	3.5.22.20
Symbol configuration object	4.7.0.0 (library 4.6.0.0)
Factory I/O	v2.5.10 Ultimate Edition
OPC interface	OPC DA / OPC UA client

APPENDIX B FIGURE PROVENANCE

All photographic and screenshot figures are frames taken from the three demonstration recordings listed below, extracted at the full capture resolution of 1872×798 . Screenshot crops of the CODESYS editors and the OPC client were upscaled with a Lanczos filter to preserve text legibility; no figure has been otherwise edited. The ladder diagram of Fig. 3 was redrawn from the online view.

- Case Study I: <https://youtu.be/-FCAna-uD4I>
- Case Study II: <https://youtu.be/kP9y-KGVdMQ>
- Case Study III: <https://youtu.be/s-syDBKdoRM>

REFERENCES

- [1] International Electrotechnical Commission, *IEC 61131-3: Programmable Controllers — Part 3: Programming Languages*, 3rd ed. Geneva, Switzerland: IEC, 2013.
- [2] CODESYS Group, *CODESYS Development System V3.5 Online Help*. Kempten, Germany: CODESYS GmbH, 2024. [Online]. Available: <https://help.codesys.com>
- [3] Real Games, *Factory I/O User Manual*, v2.5. Braga, Portugal: Real Games Ltd., 2023. [Online]. Available: <https://docs.factoryio.com>
- [4] OPC Foundation, *OPC Unified Architecture Specification, Part 1: Overview and Concepts*, Release 1.05. Scottsdale, AZ, USA: OPC Foundation, 2022.
- [5] B. Riera and B. Vigário, “HOME I/O and FACTORY I/O: a virtual house and a virtual plant for control education,” *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 9038–9043, 2017.
- [6] K.-H. John and M. Tiegkamp, *IEC 61131-3: Programming Industrial Automation Systems*, 2nd ed. Berlin, Germany: Springer, 2010.